target_setter Documentation

Release 1.0

Chheng Lydiya

Feb 23, 2026

CONTENTS:

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Objective . . . 2
- 1.3 Scope of work . . . 3

2 Hardware Specification **5**
- 2.1 Robot platform . . . 5
- 2.2 Sensors . . . 5
- 2.3 Drive system . . . 5
- 2.4 Sensors . . . 5
- 2.5 Compute hardware . . . 6
- 2.6 Network infrastructure . . . 6
- 2.7 Power system . . . 6
- 2.8 Safety constraints . . . 6
- 2.9 Notes . . . 6

3 Installation and Configuration **7**
- 3.1 Prerequisites . . . 7
- 3.2 Workspace Setup . . . 7
- 3.3 Configuration . . . 8
- 3.4 Running the Application . . . 8
- 3.5 Common Setup Pitfalls . . . 8

4 Technical Reference **9**
- 4.1 Architecture . . . 9
- 4.2 Waypoint queue and state machine . . . 11
- 4.3 State machine . . . 11
- 4.4 Communication protocol . . . 12
- 4.5 Coordinate convention and control logic . . . 14
- 4.6 Control logic . . . 15
- 4.7 Path planning . . . 16
- 4.8 Odometry handling . . . 17
- 4.9 Inverse Kinematic . . . 18

5 Troubleshooting Guide **19**
- 5.1 Communication issues . . . 19
- 5.2 Motor/driver issues . . . 19
- 5.3 Sensor issues . . . 20
- 5.4 App issues . . . 20
- 5.5 Safety/emergency stop . . . 20
- 5.6 PD controller/motion control . . . 21
- 5.7 Low-level hardware checks . . . 21

6 Appendix A: Object-Oriented Programming **23**
- 6.1 Overview . . . 23

6.2 Core concepts . . . 23
6.3 Composition over inheritance . . . 24
6.4 SOLID principles . . . 24
6.5 Design patterns . . . 24
6.6 Typical pitfalls . . . 24
6.7 Applied to robotics software . . . 24
6.8 Example: interface-driven control . . . 25

7 Appendix B: MVC Architecture **27**
7.1 Overview . . . 27
7.2 Roles . . . 27
7.3 Typical flow . . . 27
7.4 Benefits . . . 28
7.5 Tradeoffs . . . 28
7.6 Applied to a robotics UI . . . 28
7.7 Implementation hint . . . 28

8 Appendix C: MVVM Architecture **29**
8.1 Overview . . . 29
8.2 Roles . . . 29
8.3 Data binding . . . 29
8.4 Benefits . . . 30
8.5 Tradeoffs . . . 30
8.6 Applied to target setting . . . 30

9 Appendix D: ROS2 Overview **31**
9.1 What is ROS2 . . . 31
9.2 Key concepts . . . 31
9.3 QoS (Quality of Service) . . . 32
9.4 Packages and workspaces . . . 32
9.5 Lifecycle management . . . 32
9.6 Why ROS2 for robotics . . . 32

10 Appendix E: Distributed Systems **33**
10.1 Overview . . . 33
10.2 Key challenges . . . 33
10.3 Time and ordering . . . 33
10.4 CAP tradeoff . . . 34
10.5 Patterns . . . 34
10.6 Applied to this project . . . 34

11 Appendix F: Monolithic Architecture **35**
11.1 Overview . . . 35
11.2 Benefits . . . 35
11.3 Drawbacks . . . 35
11.4 Modular monolith . . . 35
11.5 Applied to robotics tooling . . . 35

12 Appendix G: ROS2 Architecture **37**
12.1 Layered structure . . . 37
12.2 Executors . . . 37
12.3 Composition . . . 37
12.4 Lifecycle nodes . . . 37
12.5 DDS details . . . 37
12.6 Best practices . . . 38

13 Appendix H: UDP **39**
13.1 Overview . . . 39
13.2 Characteristics . . . 39

13.3 Packet size and MTU . . . 39
13.4 Checksums . . . 39
13.5 Best practices for control systems . . . 39
13.6 Applied to target setting . . . 40

14 Appendix I: Quaternions **41**
14.1 Overview . . . 41
14.2 Unit quaternions . . . 41
14.3 Quaternion multiplication . . . 41
14.4 Axis-angle conversion . . . 41
14.5 Yaw extraction (2D robots) . . . 41
14.6 Numerical stability . . . 42
14.7 Applied to odometry . . . 42

15 Appendix J: Kinematics **43**
15.1 Overview . . . 43
15.2 Forward kinematics . . . 43
15.3 Inverse kinematics . . . 43
15.4 Coordinate frames . . . 43
15.5 Discretization . . . 43
15.6 Applied to this system . . . 44

16 Appendix K: State Machine **45**
16.1 Overview . . . 45
16.2 Key terms . . . 45
16.3 Design guidelines . . . 45
16.4 Hierarchical state machines . . . 45
16.5 Applied to navigation . . . 45

17 Appendix L: PID Control **47**
17.1 Overview . . . 47
17.2 PID equation . . . 47
17.3 Discrete form . . . 47
17.4 Tuning . . . 47
17.5 Anti-windup . . . 47
17.6 Filtering . . . 48
17.7 Applied to robot motion . . . 48

18 Appendix M: Localization **49**
18.1 Overview . . . 49
18.2 Sources of localization . . . 49
18.3 Probabilistic view . . . 49
18.4 Filters . . . 49
18.5 Frames . . . 49
18.6 Applied to this system . . . 50

19 Appendix N: Odometry **51**
19.1 Overview . . . 51
19.2 Kinematic integration . . . 51
19.3 Error sources . . . 51
19.4 Mitigation . . . 51
19.5 Applied to this project . . . 51

20 Appendix O: Waypoint Navigation **53**
20.1 Overview . . . 53
20.2 Path planning . . . 53
20.3 Arrival detection . . . 53
20.4 Handling edits . . . 53
20.5 Return behavior . . . 53

20.6 Applied to this system . . . 54

21 Appendix P: Complementary Filtering **55**
21.1 Overview . . . 55
21.2 Basic equation . . . 55
21.3 Why it works . . . 55
21.4 Discrete implementation . . . 55
21.5 Applied to derivative terms . . . 55
21.6 Applied to waypoint updates . . . 55

22 Appendix Q: CAN Bus **57**
22.1 Overview . . . 57
22.2 Frame structure . . . 57
22.3 Arbitration . . . 57
22.4 Bitrate and timing . . . 57
22.5 Error handling . . . 57
22.6 Classic CAN vs CAN FD . . . 58
22.7 Applied to robotics . . . 58

23 Indices and tables **59**

CHAPTER

ONE

INTRODUCTION

1.1 Background

1.1.1 Motivation

Following the previous year in Android application development for robotics system, we have been using Android app as the controller for the robots.

However, this approach works with the xbox gamepad that acts as the main controller for our robots, using the buttons and joystick to navigate the robots manually. While this method provides real-time control, it limits automation and precise waypoint navigation. The initial idea of this project originated from Mr.PHOLL Chanphayuk, who proposed an Android-based interface that allows users to interact with the robot through touch inputs on the screen, combined with odometry feedback and a virtual joystick. The concept was intended for interactive tasks, such as robot basketball playing.

After changes in term members and project direction, the focus of the project evolved from manual joystick control to a waypoint-based navigation system. This shift was encouraged by Dr.SRANG Sarot, aiming to reduce dependency on gamepad controllers and to explore a more autonomous navigation approach using Android applications as the main interface.

1.1.2 Problem Statement

Manual control using gamepads or joysticks presents several limitations in robotic navigation systems:

- Continuous use input is required, increasing operator workload
- Precise positioning is difficult due to human reaction time and control sensitivity
- The system lacks scalability for autonomous behaviors

To address these issues, a waypoint navigation system is proposed, where uses can define target positions (waypoints) through an Android application, and the robot navigates to these positions using odometry and orientation data.

However, implementing such a system introduces challenges, including:

- Accurate coordinate transformation between Android interface and the robot's reference frame
- Proper handling of robot orientation (yaw) in navigation calculations
- Reliable communication between the Android application and the robot's ROS2-based control system
- Managing waypoint editing, execution, and return behaviors without causing state conflicts

These challenges motivated the need to redesign the calculation logic and communication protocol to ensure compatibility with ROS2 and improve navigation accuracy.

1.1.3 Literature Review

Waypoint-based navigation is a common approach in mobile robotics, especially in autonomous systems using frameworks such as ROS and ROS2. Previous works and standard robotics literature describe waypoint navigation as a method where a robot moves through a sequence of pre-defined target positions using localization and motion control algorithms.

Odometry-based navigation is widely used for estimating a robot's position by integrating wheel encoder data over time. Although odometry is simple to implement, it is sensitive to drift and orientation errors, particularly in yaw estimation. Many systems combine odometry with higher-level navigation logic to mitigate these issues.

Mobile interfaces for robot control, including Android-based applications, have also been explored as flexible and user-friendly solutions. Touch-based interfaces allow intuitive interaction, but require careful mapping between screen coordinates and real-world robot motion.

This project builds upon these established concepts, focusing on practical implementation rather than theoretical optimization.

1.1.4 Concepts

The waypoint navigation system in this project is based on the following key concepts:

- Waypoint navigation: the robot is instructed to move toward pre-defined target positions instead of being continuously controlled by the user.
- Odometry: the robot's position and orientation are estimated using incremental motion data.
- Yaw orientation: the robot's heading angle is critical for accurate navigation and must be correctly handled in coordinate transformations.
- Client-sever communication: the Android application acts as a client that sends waypoints data to the robot, while the robot processes these commands within a ROS2 environment.

For simplification, the system assumes:

- A planar (2D) environment
- Reliable odometry data without sensor fusion
- Pre-defined motion control handled by the robot's existing control stack

The Android application focuses on target setting, waypoint management, and command transmission, while the robot handles low-level motion execution.

1.2 Objective

This project is developed in order to:

- Design and implement Android app to define, edit, and manage navigation waypoints on a pre-defined map
- Develop custom ROS2 control logic to receive waypoint data and execute navigation commands on the robot
- Implement accurate coordinate transformation and scaling between Android interface and the robot's reference frame
- Handle robot orientation (yaw) correctly during waypoint navigation to improve positioning accuracy
- Establish reliable client-server communication between the Android app and the ROS2-based robot system
- Implement waypoint execution features including sequential navigation, editing, and return-to-previous-waypoint behaviors
- Test and evaluate system performance, identifying limitations related to map alignment, yaw accuracy, and state management.

1.3 Scope of work

This project covers:

- Android application development (Target_Setter)
- Communication system
- Robotic software development (ROS2)
- System integration and testing

This project does not include: - Full autonomous robot navigation - Sensor fusion beyond basic odometry and IMU integration - Algorithm optimization - Low-level communication protocol - Hardware-level motor driver design - Network security mechanism

CHAPTER

TWO

HARDWARE SPECIFICATION

This section outlines the hardware components and specifications for the mobile robot platform used in this project. The hardware selection is based on the requirements for waypoint navigation, odometry estimation, and client-server communication with an Android application.

2.1 Robot platform

- **Type:** omnidirectional mobile robot with mecanum wheels (holonomic motion)
- **Dimensions:** 0.5 m x 0.5 m x 0.3 m (approximately)

2.2 Sensors

The robot is equipped with 4 independent DC motors, driving 4 mecanum wheels with encoders attached for feedback. To compenstate for yaw estimation, an IMU (Inertial Measurement Unit) is included, providing angular velocity and linear acceleration data.

2.3 Drive system

- **Motors:** 4 DC motors, each driving on mecanum wheel
- **Motor specifications:** 24 V, 200 W max, integrated encoders
- **Motor drivers:** custom-built smart driver boards, each supporting 2 motors
- **Wheel type:** mecanum 0.06 radius (approximately)
- **Feedback:** quadrature encoders for wheel odometry

2.4 Sensors

- **Inertial Measurement Unit (IMU):** provides angular velocity and linear acceleration - **Model:** HFI-A9 - **Sampling rate:** 100 Hz
- **Encoder:** integrated with each motor; used for odometry estimation

2.5 Compute hardware

- **High-level processor:** NVIDIA Jetson Xavier - **OS:** Ubuntu 20.04 - **Middleware:** ROS2 Foxy - **Cooling:** cooling fan
- **Low-level controller:** STM32 microcontroller - Interfaces with motor driver and encoders for real-time control
- **Communication:** CAN bus at 115200 bps

2.6 Network infrastructure

- **High-level control:** UDP of Wi-Fi between Android application and Jetson
- **IP configuration:** dynamic IP over DHCP
- **Indoor deployment:** Wi-Fi network required
- **Security:** basic
- **Remote access:** SSH for development and debugging purposes

2.7 Power system

- **Motors:** 24 V Li-ion battery pack, DC-DC converter (12 V) for motor drivers
- **Electronics:** 12 V Li-ion battery pack, voltage regulator for Jetson, sensors, and STM32
- **Safety features:** - Fuses on smart driver boards - Overcurrent protection - Emergency stop push button

2.8 Safety constraints

- **hardware-level safety:** fuses and emergency stop
- **Software-level safety:** - Speed limits enforced in control software - Communication failure handling - Watchdog timers for microcontroller to prevent system hang

2.9 Notes

1. Ensure CAN bus termination is correct to prevent communication errors
2. Confirm battery voltages before operation; over/under-voltage may damage electronics
3. When adding additional sensors, update ROS2 launch files for proper integration
4. For software testing, always use the emergency stop and verify motor driver fuses

CHAPTER

THREE

INSTALLATION AND CONFIGURATION

This section provides a comprehensive guide to installing and configuring the ROS2-based mobile manipulation app. It covers the necessary prerequisites, workspace setup, build instructions, configuration parameters, and tips for running the application effectively.

3.1 Prerequisites

- ROS2 installed on Ubuntu (any version is fine)
- Jetson Nano setup with ROS2 and necessary drivers for the robot hardware
- Android Studio installed for app development
- Basic familiarity with ROS2 concepts and Android development
- Network configuration for communication between the Android app and the robot
- Python 3 and necessary ROS2 Python packages for the robot nodes

3.2 Workspace Setup

1. Create a ROS2 workspace on the Jetson Nano:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
source install/setup.bash
```

2. Clone the project repository into the `src` directory:

```
cd ~/ros2_ws/src
git clone <repository_url>
```

3. Build the workspace:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

4. Set up the Android project in Android Studio by importing the project from the cloned repository.

3.3 Configuration

- Update the ROS2 node parameters in the configuration files located in the *config* directory of the ROS2 package. This includes setting the correct IP address for communication and any robot-specific parameters.
- Ensure that the Android app's network settings are configured to communicate with the robot's IP address
- Adjust the waypoint navigation parameters such as speed limits, waypoint tolerance, and control gains as needed for your specific robot and environment.

3.4 Running the Application

1. Start the ROS2 nodes on the Jetson Nano:

```
source ~/ros2_ws/install/setup.bash
ros2 run <package_name> <package_node> # repeat for all necessary nodes
```

2. Run the Android application from Android Studio on a connected device or emulator. Or you can just open the `.apk` file on the phone directly.
3. Use the app interface to set waypoints and observe the robot's behavior as it navigates to the specified locations.

Attention: This release doesn't have launch file yet. Launch file will be added in the next update release.

3.5 Common Setup Pitfalls

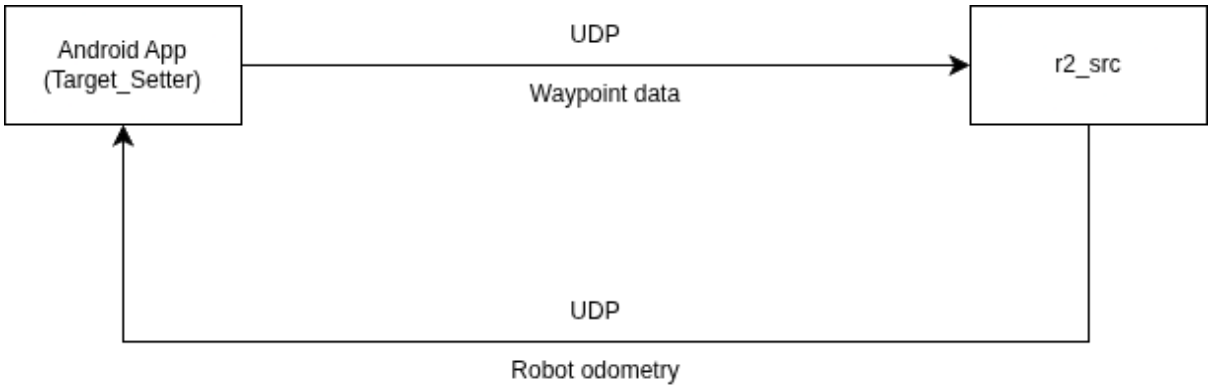
- Incorrect IP address configuration leading to communication failures between the app and the robot.
- Missing dependencies or incorrect ROS2 environment setup on the Jetson Nano.
- Incompatible ROS2 versions between the robot nodes and the app's communication protocol.
- Issues with the robot's hardware drivers or sensor configurations that affect navigation performance.
- Network connectivity issues, such as firewall settings or Wi-Fi interference, that disrupt communication between the app and the robot.

CHAPTER

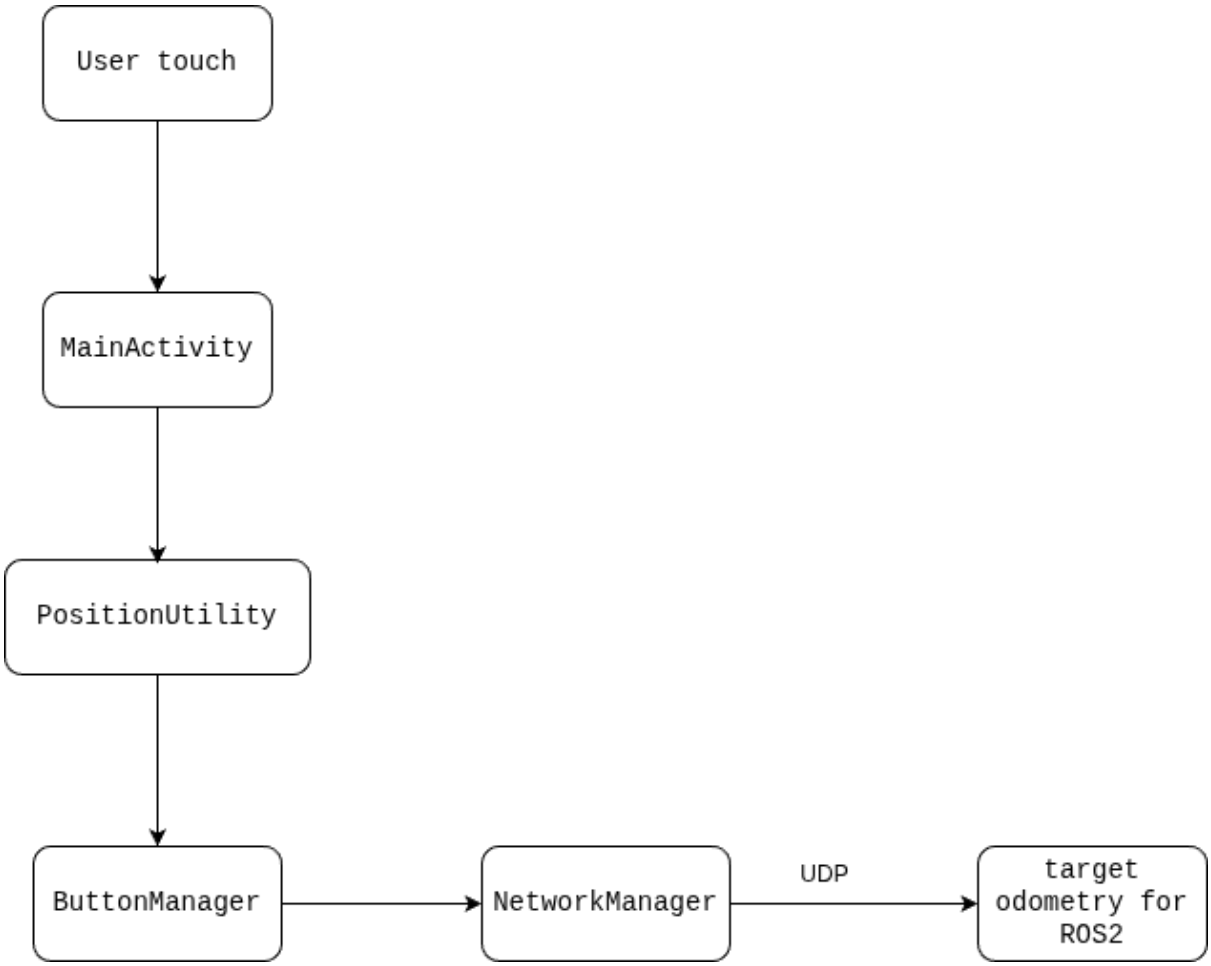
FOUR

TECHNICAL REFERENCE

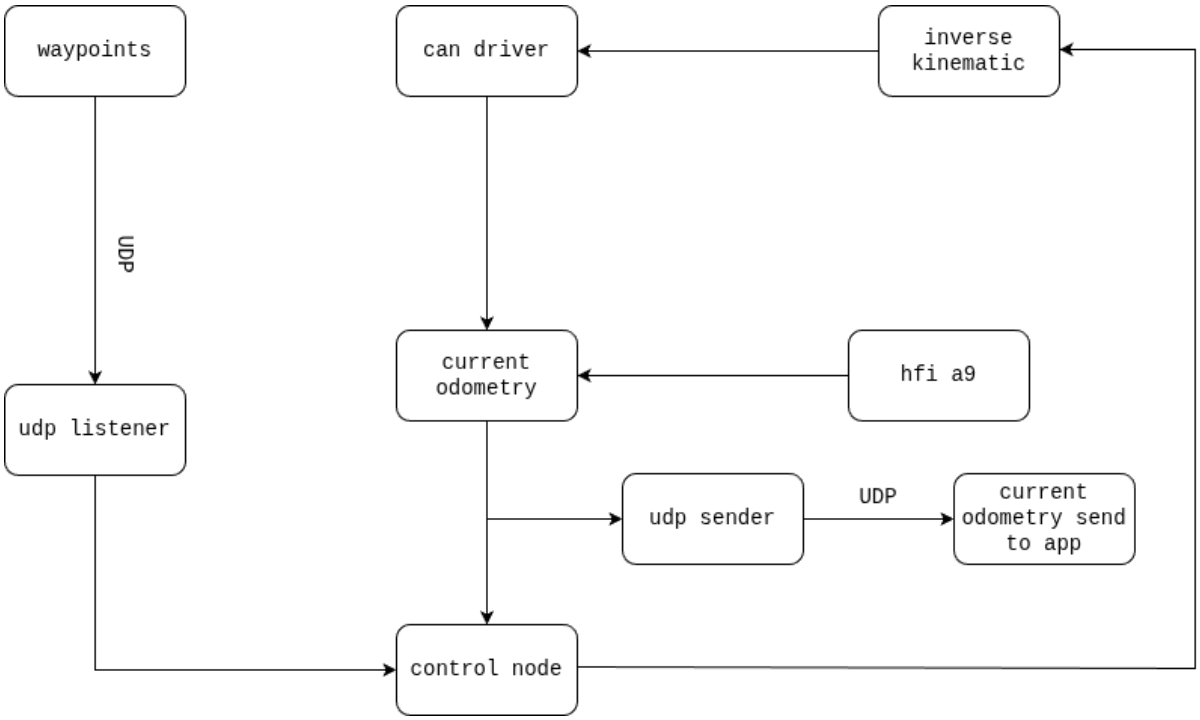
4.1 Architecture



4.1.1 Target_Setter app



4.1.2 r2_src



4.2 Waypoint queue and state machine

4.2.1 Waypoint queue

- **Waypoint queue:** stores received waypoints from */waypoint*
- **Active target:** currently executing waypoint
- **Visited waypoints:** stack of completed waypoints for return

4.3 State machine

- `IDLE`: no active target, waiting for waypoint or return command
- `NAVIGATING`: moving to `active_target`
- `EDIT`: active target is being edited
- `PAUSED`: robot is paused by keyboard or automatic pause
- `RETURNING`: moving to last visited waypoint (return command)

4.4 Communication protocol

4.4.1 UDP packet structure

Waypoints packet

- **Purpose:** send a list of waypoints for the robot to follow
- **Frequency:** send when pressing Send in the app

Packet Structure (binary):

Offset	Size (bytes)	Description
0	4	Header (0xAA)
4	4	Number of waypoints (counter)
8	4	Plan ID (version)
12	16 * N	Waypoint (X, Y) in double (8 B)

- N = number of waypoints
- Coordinates are in meters, converted from percentage position
- Byte order: Little Endian

Edit waypoint packet

- **Purpose:** update a previously sent waypoint
- **Frequency:** send when user edits a waypoint via the app

Packet structure (binary):

Offset	Size (bytes)	Description
0	1	Edited flag (1 = edited)
1	4	Waypoint index (1-based)
5	4	Plan ID
9	8	New X coordinate (double)
17	8	New Y coordinate (double)

Return message packet

- **Purpose:** send a single boolean signal to the robot
- **Frequency:** send when return button is pressed

Packet structure (binary):

Offset	Size (bytes)	Description
0	1	Edited flag (0=false,1=true)

Odometry data reception

- **Purpose:** receives current robot odometry
- **Frequency:** 10Hz

Packet structure (binary):

Offset	Size (bytes)	Description
0	8	X position (double)
8	8	Y position (double)
16	8	Yaw (double, radians)

- Byte order: Little Endian
- Latest odometry stored in memory for waypoint calculations

4.4.2 UDP data flow

Sending data

- App uses `NetworkManager.sendWaypoints()`, `sendEdit()`, and `sendMesage()`
- Waypoints are converted to binary packets before sending
- Each send operation is done in a separate thread
- Plan ID differentiates waypoint plans
- Packets sent via UDP to robot IP/port (default 5050)

Receiving data

- Persistent UDP socket listens (`startReceiving(port)`)
- Robot sends odometry at 10Hz (X, Y, Yaw)
- Incoming packets converted using Little Endian byte order
- Latest odometry stored in `latestOdometry`

Processing data

When adding waypoint:

- Convert touch position to normalized coordinates
- Compute target X, Y in meters
- Add to waypoint list, update UI
- Send waypoint via UDP

When editing waypoint: - Specify index and new coordinates - Update list and redraw UI - Send edit packet via UDP with plan ID

Return message: - Single-byte packet sent to trigger return behavior

Odometry: - Display robot position - Compute accurate target positions

4.5 Coordinate convention and control logic

4.5.1 Coordinate flow

Touch → UI → App → Robot

```
Touch Event (screen pixels)
   ↓ absolutePosition()
Absolute Coordinates relative to boundaryView
   ↓ normalizedPosition()
Normalized Coordinates (0-1)
   ↓ percentagePosition()
Percentage Coordinates (0-100%)
   ↓ targetPosition()
Global Frame (meters) → UDP → Robot
```

Robot → App (Odometry)

```
Odometry Packet (X, Y, yaw in meters) → inputPosition()
   ↓ metersToPxX/Y
Pixels on UI → Display robot position
```

4.5.2 Frame definition

Local frame:

- Origin: the center of the robot
- +x: forward
- +y: left
- +yaw: counter-clockwise

Global frame: - Origin: top-left of the screen on the target_setter app - +x: rightward on the screen on the target_setter app - +y: downward on the screen on the target_setter app - Waypoints that is sent to the robot is in global frame

Units:

On the robot and the data sent from app:

- x: meters
- y: meters
- yaw: radians (data from app only send x and y, yaw must be infer from them)
- Vx: m/s
- Vy: m/s
- Wz: rad/s

On the app: - x: percentage - y: percentage

4.5.3 Touch input conversion

Absolute position (touch):

```
double absoluteX = rawX - boundaryLeft;
double absoluteY = rawY - boundaryTop;
```

- Converts raw screen coordinate to coordinate relative to *boundaryView*
- Clamped to boundary edges

Normalized position (0-1):

```
double normalizedX = absoluteX / screenWidth;
double normalizedY = absoluteY / screenHeight;
```

- Represents position as fraction of the screen

Percentage position (0-100%):

```
double percentageX = normalizedX * 100.0;
double percentageY = normalizedY * 100.0;
```

- Used as intermediate format for calculation into world meters
- Checked against 0-100% boundary

Global meters (for UDP):

```
double targetX = (percentageX / 100.0) * GAME_FIELD_X;
double targetY = (percentageY / 100.0) * GAME_FIELD_Y;
```

- Converts to meters based on field dimension
- Result is sent to robot

Robot odometry (UI display):

```
double inputX = odometryData[0] * (screenWidth / GAME_FIELD_X);
double inputY = odometryData[1] * (screenHeight / GAME_FIELD_Y);
```

- Converts meters to pixels for UI display
- Clamped boundary using *validateBoundary()*

4.6 Control logic

4.6.1 PD controller

- **Position errors:**

computed in global frame

```
current_x_error_g = self.desired_x - self.current_x
current_y_error_g = self.desired_y - self.current_y

current_yaw_p_error = self.normalize_angle(self.desired_yaw - self.current_yaw)
```

- **Derivative errors:**

```
current_yaw_d_error = (current_yaw_p_error - self.previous_yaw_p_error)/self.dt
current_x_d_error = (current_x_error_g - self.previous_x_p_error)/self.dt
current_y_d_error = (current_y_error_g - self.previous_y_p_error)/self.dt
```

- **Complementary filtering:**

apply complementary filter on D term to smooth the movement

```
current_x_d_error = ALPHA_D * current_x_d_error + (1 - ALPHA_D) * self.previous_x_d_
↪error
current_y_d_error = ALPHA_D * current_y_d_error + (1 - ALPHA_D) * self.previous_y_d_
↪error
current_yaw_d_error = ALPHA_D * current_yaw_d_error + (1 - ALPHA_D) * self.previous_
↪yaw_d_error
```

- **PD controller:**

```
linear_vel_x_g = self.k_p_linear * current_x_error_g + self.k_d * current_x_d_error
linear_vel_y_g = self.k_p_linear * current_y_error_g + self.k_d * current_y_d_error
angular_vel_z_g = self.k_p_yaw * current_yaw_p_error + self.k_d_yaw * current_yaw_d_
↪error
```

- **Frame conversion:**

```
linear_vel_x_l =  math.cos(self.current_yaw) * current_x_error_g + math.sin(self.
↪current_yaw) * current_y_error_g
linear_vel_y_l = -math.sin(self.current_yaw) * current_x_error_g + math.cos(self.
↪current_yaw) * current_y_error_g
```

- Units must match: meters (x, y), radians (yaw), m/s (velocities)
- Linear velocity in robot frame, angular velocity around z-axis
- Low-pass filtering applied on derivative term
- Return *linear_vel_x*, *linear_vel_y*, and *angular_vel_z*

4.6.2 Dead reckoning

If the velocity is less than error threshold, overwrite it with 0

4.6.3 Speed limit

If velocity exceeds maximum speed, overwrite it with maximum speed

4.7 Path planning

4.7.1 Arrivel detection

- Target reached if: - Distance < `goal_tolerance_pos = 0.05m` - Remains withitn tolerance for `0.5s`
- Arrival time accumulated using `dt` from timer callback

4.7.2 Waypoint callback

- Store each waypoint in a dictionary
- Append it into the queue
- Copy the waypoint in the queue to `active_target` to execute

4.7.3 Waypoints execution

- if `active_target` does not exist, and there's waypoint in the queue, pop the queue, then reset robot state to go to the waypoint that's being popped
- if return is triggered, and there's visited waypoint, command the robot to go the visited waypoint
- Otherwise, stop the robot

4.7.4 Waypoint update

- Store the edited waypoint in a dictionary
- If the edited waypoint is active, use complementary to smooth the robot movement when update to prevent jumping suddenly

```
self.active_target['x'] = (1 - ALPHA) * self.active_target['x'] + ALPHA * update_wp[
↪'updated_x']
self.active_target['y'] = (1 - ALPHA) * self.active_target['y'] + ALPHA * update_wp[
↪'updated_y']
```

- Otherwise, just update the waypoint as normal

4.7.5 Pause

- Robot pauses motion at each target
- If distance from the robot to the target is less than tolerance, stop the robot
- If the elapsed time during pause exceeds pause duration, command the robot to go to the next waypoint

4.7.6 Return

- Update the return flag

4.8 Odometry handling

4.8.1 Frame conversion

- Calculate velocity in local frame

```
linear_vel_x_local = (self.R/4) * (self.motor_vel[0] + self.motor_vel[1] + self.motor_
↪vel[2] + self.motor_vel[3])
linear_vel_y_local = (self.R/4) * (-self.motor_vel[0] + self.motor_vel[1] + self.
↪motor_vel[2] - self.motor_vel[3])
w_z = (self.R * (-self.motor_vel[0] + self.motor_vel[1] - self.motor_vel[2] + self.
↪motor_vel[3]))/(2 * (self.l_x + self.l_y))
```

- Convert them to global frame

```
v_x_global = math.cos(self.current_yaw) * linear_vel_x_local - math.sin(self.current_
↪yaw) * linear_vel_y_local
v_y_global = math.sin(self.current_yaw) * linear_vel_x_local + math.cos(self.current_
↪yaw) * linear_vel_y_local
```

- Integrate velocity to get position

```
self.x += v_x_global * self.dt
self.y += v_y_global * self.dt
```

4.8.2 Offset

```
self.x_odom = (self.x - self.x_start) - self.frame_offset_x
self.y_odom = (self.y - self.y_start) - self.frame_offset_y
```

4.8.3 Yaw

- Calculate yaw from quaternion

```
def calculate_yaw(self, q_x, q_y, q_z, q_w):
    siny_cosp = 2 * (q_w * q_z + q_x * q_y)
    cosy_sinp = 1 - 2 * (q_y**2 + q_z**2)
    return math.atan2(siny_cosp, cosy_sinp)
```

- Normalize yaw

```
def normalize_angle(self, a):
    return math.atan2(math.sin(a), math.cos(a))
```

- Offset yaw

```
self.current_yaw = (self.yaw - self.yaw_start) + 2 * math.pi * self.overflow_counter
```

- Convert to quaternion to publish

```
def publish_yaw(self, yaw):
    q = Quaternion()
    q.w = math.cos(yaw/2)
    q.x = 0.0
    q.y = 0.0
    q.z = math.sin(yaw/2)
    return q
```

4.9 Inverse Kinematic

```
V_wheel = [
Vx/self.R - Vy/self.R - omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R + Vy/self.R + omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R + Vy/self.R - omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R - Vy/self.R + omega_z*(self.lx + self.ly)/(2.0*self.R)
]
```

CHAPTER

FIVE

TROUBLESHOOTING GUIDE

5.1 Communication issues

5.1.1 Symptom: robot doesn't repond to app command

Wi-Fi connectivity issues:

- Check that the robot is connected to the correct Wi-Fi network and has a valid IP address
- Verify IP address of the robot matches the one configured in the app
- Use `ifconfig` on the Jetson to check network interfaces and IP configuration

UDP packets not received:

- Confirm the correct port (default `5050`) is used
- Ensure packets are sent in **Little Endian** format

Firewall blocking UDP:

- Disable firewall or allow UDP traffic on port 5050

5.1.2 Symptom: waypoints not updating/edits ignored

Possible causes and solutions:

- Plan ID mismatch between app and robot
- Edited waypoint index out of range
- Packet corruption: check Little Endian encoding and packet size

5.2 Motor/driver issues

5.2.1 Symptom: robot does not move or moves incorrectly

Possible causes and solutions:

- Motor driver fuse blown: replace fuse and test
- Motors not powered: verify 24 V motor battery and DC-DC converter
- Encoder feedback lost: check encoder connections to STM32
- Wheels misaligned: check mecanum wheel orientation
- Incorrect velocities from controller: verify PD controller gains

5.2.2 Symptom: robot moves jerkily or overshoots targets

- Derivative term too noisy: verufy `ALPHA_D` filtering
- Low update frequency: check odometry reception at 10 Hz
- Active waypoint updates cause jumps: ensure complementary smoothing is applied

5.3 Sensor issues

5.3.1 Symptom: odometry is incorrect/robot drifts

- Checks IMU calibration and orientation
- Verify encoder counts correspond to expected wheel rotation
- Confirm yaw offset calculation (`yaw_start`, `overflow_counter`)
- Dead reckoning integration might accumulate error: check `dt` values

5.3.2 Symptom: robot reports zero movement despite motor command

- Motor velocities not reaching STM32: verify CAN bus communication
- Verify low-level motor control code maaps velocities correctly to wheels

5.4 App issues

5.4.1 Symptom: robot position not displayed correctly

- Verify coordinate conversions from meters to pixels (`metersToPxX/Y`) are correct
- Check `latestOdometry` is being updated from UDP socket
- Confirm screen boundary and field dimensions (`GAME_FIELD_X/Y`) match robot workspace

5.4.2 Symptom: touch input does not map to correct target

- Check touch, normalized, percentage, global meter conversion
- Ensure `boundaruView` dimensions match the UI

5.5 Safety/emergency stop

5.5.1 Symptom: robot does not stop when emergency button pressed

- Verify emergency stop wiring and fuses are intact
- Ensure software checks for emergency stop signal are active
- Test manually by cutting power to confirm hardware response

5.6 PD controller/motion control

5.6.1 Symptom: robot oscillates around waypoint

- Decreases `k_p_linear` or `k_p_yaw` gains
- Increase derivative damping (`ALPHA_D`)
- Verify derivative errors are computed correctly using `dt`

5.6.2 Symptom: robot fails to reach waypoint

- Check `goal_tolerance_pos` and pause duration
- Confirm `active_target` queue is populated and not empty
- Make sure velocity limits allow the robot to reach target within constraints

5.7 Low-level hardware checks

- **Jetson:** check power LED, CPU usage, and fan
- **STM32:** verify firmware is running; check CAN message counts
- **Batteries:** measure voltage under load; replace if below threshold
- **Motor drivers:** check for overcurrent events or thermal shutdowns

Attention:

1. Always start troubleshooting from connectivity, sensors, motion, control logic
2. Use logs from app (`sendWaypoints()`, `latestOdometry`) to verify packet flow
3. Test motor and wheel functionality before running autonomous commands
4. Keep fuses, emergency stop, and emergency protocols ready during testing

CHAPTER

SIX

APPENDIX A: OBJECT-ORIENTED PROGRAMMING

6.1 Overview

Object-oriented programming (OOP) models software as interacting objects that encapsulate state and behavior. The goal is to make programs easier to reason about by grouping related data and functions and enforcing clear boundaries.

6.2 Core concepts

6.2.1 Classes and objects

- A class defines a blueprint for data (fields) and behavior (methods).
- An object is an instance of a class created at runtime.

6.2.2 Encapsulation

- Hide internal details and expose a minimal public interface.
- Prevents accidental misuse and reduces coupling.

6.2.3 Abstraction

- Present only essential features and ignore implementation details.
- Encourages focusing on what an object does, not how it does it.

6.2.4 Inheritance

- A subclass derives from a base class and can extend or override behavior.
- Supports reuse but can introduce tight coupling if overused.

6.2.5 Polymorphism

- Code can work with objects through a shared interface.
- Allows different implementations to be swapped with minimal changes.

6.3 Composition over inheritance

Composition builds complex behavior by combining smaller objects, which helps avoid fragile inheritance hierarchies. When possible, prefer composition and interfaces to keep dependencies explicit.

6.4 SOLID principles

- Single Responsibility: one reason to change.
- Open/Closed: open for extension, closed for modification.
- Liskov Substitution: subclasses must be usable where base classes are expected.
- Interface Segregation: small, focused interfaces.
- Dependency Inversion: depend on abstractions, not concrete types.

6.5 Design patterns

Common patterns that align with OOP:

- Strategy: swap algorithms without changing the caller.
- Observer: notify subscribers on state changes.
- Command: encapsulate an action as an object.
- Factory: centralize object creation.

6.6 Typical pitfalls

- Deep inheritance hierarchies that obscure behavior.
- God objects that violate single responsibility.
- Exposing internal state directly, making invariants hard to enforce.

6.7 Applied to robotics software

In robotics systems, OOP helps isolate concerns like communication, control, and sensing. For example:

- A network manager can encapsulate UDP send/receive behavior.
- A controller class can wrap control logic and state updates.
- A UI layer can treat the model as a stable interface and remain decoupled from data sources.

6.8 Example: interface-driven control

```
class VelocityController:
    def compute(self, target, state):
        raise NotImplementedError

class PDController(VelocityController):
    def compute(self, target, state):
        # PD math here
        return vx, vy, wz

def step(controller, target, state):
    return controller.compute(target, state)
```

This keeps the control pipeline stable while allowing multiple controllers to be used during testing or future upgrades.

CHAPTER

SEVEN

APPENDIX B: MVC ARCHITECTURE

7.1 Overview

Model-View-Controller (MVC) separates data, presentation, and user input into three roles. This separation improves maintainability by reducing coupling.

7.2 Roles

7.2.1 Model

- Owns the application state and business rules.
- Exposes operations that modify data in a controlled way.

7.2.2 View

- Renders the model state to the user.
- Should be mostly passive and not contain business logic.

7.2.3 Controller

- Interprets user input and updates the model.
- Chooses which view to refresh or navigate to.

7.3 Typical flow

1. User interacts with the View.
2. Controller translates the action into model updates.
3. Model updates its state and notifies views.
4. View re-renders using the new model state.

7.4 Benefits

- Clear separation of responsibilities.
- Easier testing of logic by isolating the model.
- Views can be swapped without changing the model.

7.5 Tradeoffs

- Controllers can grow large if not structured.
- Requires discipline to keep business logic in the model.

7.6 Applied to a robotics UI

In a waypoint-setting UI:

- Model holds the waypoint list, active target, and plan ID.
- View renders the field, robot position, and waypoint markers.
- Controller translates touch events into model updates and network sends.

7.7 Implementation hint

Use small controllers to handle independent features (e.g., waypoint editing, return-to-base). This avoids a single controller that becomes a bottleneck.

CHAPTER

EIGHT

APPENDIX C: MVVM ARCHITECTURE

8.1 Overview

Model-View-ViewModel (MVVM) emphasizes data binding between view and view-model. It is common in UI frameworks with declarative views.

8.2 Roles

8.2.1 Model

- Domain data and business rules.
- Independent of UI technologies.

8.2.2 View

- UI elements and layout.
- Binds directly to view-model properties.

8.2.3 ViewModel

- Adapts model data into UI-friendly formats.
- Exposes commands or actions that the view can invoke.

8.3 Data binding

Bindings update the UI when model data changes, and optionally update the model when the user edits the UI. This reduces glue code but requires careful state management.

8.4 Benefits

- Strong separation between view and logic.
- Easier to test view-model behavior.
- Reduced UI boilerplate.

8.5 Tradeoffs

- Overuse of bindings can make data flow hard to trace.
- View-models can become too large without clear boundaries.

8.6 Applied to target setting

A view-model might expose:

- A list of waypoints as UI-friendly points.
- A selected waypoint index.
- Commands for send, edit, and return actions.

This allows the view to remain simple and focused on rendering and input.

CHAPTER

NINE

APPENDIX D: ROS2 OVERVIEW

9.1 What is ROS2

ROS2 (Robot Operating System 2) is a middleware framework for building robot applications. It provides communication, tools, and libraries for composing complex robot systems from modular nodes.

9.2 Key concepts

9.2.1 Nodes

- Independent processes that perform specific tasks.
- Nodes communicate via topics, services, and actions.

9.2.2 Topics

- Publish/subscribe channels for streaming data.
- Designed for high-frequency sensor and state updates.

9.2.3 Services

- Synchronous request/response interactions.
- Used for operations like configuration or queries.

9.2.4 Actions

- Long-running goals with feedback and cancellation.
- Suitable for navigation and planning tasks.

9.2.5 Parameters

- Runtime configuration values stored on nodes.
- Can be read and set externally.

9.3 QoS (Quality of Service)

ROS2 exposes DDS QoS settings like reliability, durability, and history depth. Selecting the right QoS is critical in lossy networks or real-time systems.

9.4 Packages and workspaces

- A package is a unit of build and deployment.
- A workspace contains multiple packages and their build artifacts.

9.5 Lifecycle management

ROS2 supports managed nodes with lifecycle states (unconfigured, inactive, active, finalized). This helps bring up systems in a predictable order.

9.6 Why ROS2 for robotics

- Scalable communication infrastructure.
- Tooling for visualization and debugging.
- Large ecosystem of drivers and algorithms.

CHAPTER

TEN

APPENDIX E: DISTRIBUTED SYSTEMS

10.1 Overview

A distributed system is a collection of independent processes that cooperate via a network. Robotics stacks are inherently distributed, with sensors, controllers, and user interfaces running on separate nodes.

10.2 Key challenges

10.2.1 Latency and jitter

- Network delays vary over time.
- Control loops must tolerate nondeterministic timing.

10.2.2 Partial failures

- Components can fail independently.
- Systems should degrade gracefully when nodes disconnect.

10.2.3 Consistency

- Shared state is hard to keep synchronized.
- Stale data can lead to incorrect decisions.

10.3 Time and ordering

- Clocks drift between machines.
- Use timestamps and sequence numbers to detect ordering issues.

10.4 CAP tradeoff

The CAP theorem highlights tradeoffs between consistency, availability, and partition tolerance. Robotics systems often prioritize availability and timely responses over strict consistency.

10.5 Patterns

- Publish/subscribe for streaming state.
- Request/response for configuration.
- Heartbeats for liveness detection.

10.6 Applied to this project

Waypoint commands and odometry updates are exchanged over UDP, which trades reliability for low latency. The app must tolerate packet loss and occasional stale data while keeping the UI responsive.

CHAPTER

ELEVEN

APPENDIX F: MONOLITHIC ARCHITECTURE

11.1 Overview

A monolithic architecture bundles application components into a single deployable unit. This contrasts with microservices or distributed components.

11.2 Benefits

- Simple deployment and debugging.
- No network overhead between internal modules.
- Easier to maintain transaction boundaries.

11.3 Drawbacks

- Tightly coupled components reduce flexibility.
- Scaling requires duplicating the entire system.
- Large codebases can slow development without strong modularity.

11.4 Modular monolith

A modular monolith preserves a single deployment but enforces internal module boundaries. This approach often provides the best of both worlds when the system does not require independent scaling.

11.5 Applied to robotics tooling

A standalone UI app can be treated as a monolith with clear internal modules for networking, input handling, rendering, and data persistence. This keeps deployment simple while still enabling maintainability.

CHAPTER

TWELVE

APPENDIX G: ROS2 ARCHITECTURE

12.1 Layered structure

ROS2 is built as a stack of layers:

- Client libraries: rclcpp (C++), rclpy (Python)
- ROS client library: rcl
- Middleware interface: rmw
- DDS implementation: Fast DDS, Cyclone DDS, or Connext

12.2 Executors

Executors schedule callbacks for subscriptions, timers, and services. Common executors include single-threaded and multi-threaded variants. Callback groups control concurrency and prevent data races.

12.3 Composition

ROS2 supports composable nodes that run in a single process. This reduces overhead and improves performance when nodes are tightly coupled.

12.4 Lifecycle nodes

Lifecycle nodes provide deterministic startup and shutdown behavior. They move through states like unconfigured, inactive, active, and finalized.

12.5 DDS details

DDS handles discovery, serialization, and transport. QoS policies determine how data is delivered and what guarantees are provided.

12.6 Best practices

- Choose QoS profiles based on network conditions.
- Use namespaces to keep large systems organized.
- Keep topics small and publish at stable rates.

CHAPTER

THIRTEEN

APPENDIX H: UDP

13.1 Overview

User Datagram Protocol (UDP) is a connectionless transport protocol that sends independent datagrams. It is fast and lightweight but provides no delivery or ordering guarantees.

13.2 Characteristics

- Connectionless and stateless.
- Packets can be dropped, duplicated, or reordered.
- Lower overhead than TCP.

13.3 Packet size and MTU

UDP payloads that exceed the path MTU may be fragmented, which increases loss risk. Design packet formats to be compact and avoid fragmentation where possible.

13.4 Checksums

UDP includes a checksum for error detection but no retransmission. Applications must decide whether to drop or tolerate corrupted packets.

13.5 Best practices for control systems

- Include sequence numbers or plan IDs to identify stale data.
- Use fixed-size binary formats for consistent parsing.
- Keep send rates bounded to avoid congestion.
- For critical commands, consider sending multiple times or adding acknowledgments.

13.6 Applied to target setting

The waypoint protocol favors low latency and simplicity. The app can tolerate occasional packet loss while maintaining responsiveness.

CHAPTER

FOURTEEN

APPENDIX I: QUATERNIONS

14.1 Overview

Quaternions represent 3D rotations without gimbal lock. A quaternion is written as $q = (w, x, y, z)$ where w is the scalar component and (x, y, z) is the vector component.

14.2 Unit quaternions

A rotation quaternion is unit length:

$$\|q\| = \sqrt{w^2 + x^2 + y^2 + z^2} = 1$$

14.3 Quaternion multiplication

Combining rotations uses quaternion multiplication, which is not commutative. If q_1 and q_2 are rotations, the composed rotation is $q = q_2 otimes q_1$.

14.4 Axis-angle conversion

For rotation angle $theta$ around unit axis $hat{u}$:

$$q = (\cos(\theta/2),\ \hat{u}_x \sin(\theta/2),\ \hat{u}_y \sin(\theta/2),\ \hat{u}_z \sin(\theta/2))$$

14.5 Yaw extraction (2D robots)

For planar motion, yaw can be extracted from a quaternion:

$$\text{yaw} = \arctan 2(2(wz + xy),\ 1 - 2(y^2 + z^2))$$

14.6 Numerical stability

- Normalize quaternions after integration.
- Avoid repeated small-angle conversions without renormalization.

14.7 Applied to odometry

Yaw is computed from quaternion orientation and then used in frame transforms for velocities and positions.

CHAPTER

FIFTEEN

APPENDIX J: KINEMATICS

15.1 Overview

Kinematics describes the relationship between motion variables without considering forces. For robots, this links wheel velocities to body motion.

15.2 Forward kinematics

Given wheel speeds, compute body velocities. For a holonomic robot, the relationship can be expressed as:

$$\mathbf{v} = \mathbf{J} \cdot \omega$$

where $mathbf{J}$ is the kinematic matrix and $mathbf{omega}$ are wheel rates.

15.3 Inverse kinematics

Given desired body velocities, compute wheel speeds:

$$\omega = \mathbf{J}^{-1} \cdot \mathbf{v}$$

If $mathbf{J}$ is not square, a pseudoinverse is used.

15.4 Coordinate frames

- Local frame: attached to the robot body.
- Global frame: fixed to the environment or map.

Transform between frames using rotation matrices derived from yaw.

15.5 Discretization

Velocities are integrated over time to compute position:

$$x_{k+1} = x_k + v_x \Delta t$$

and similarly for y and $theta$.

15.6 Applied to this system

The kinematic model converts desired planar velocities into wheel velocities and back into odometry updates.

CHAPTER

SIXTEEN

APPENDIX K: STATE MACHINE

16.1 Overview

A state machine models a system with a finite number of modes. Each state has well-defined behavior and transitions that occur on events or conditions.

16.2 Key terms

- State: a named mode of operation.
- Transition: a change from one state to another.
- Guard: a condition that must be true for a transition.
- Action: logic executed on entry, exit, or during a state.

16.3 Design guidelines

- Keep states orthogonal: one reason to change.
- Prefer explicit transitions over implicit side effects.
- Log transitions for debugging.

16.4 Hierarchical state machines

Nested states allow shared behavior at higher levels. This reduces duplication when multiple states share common logic.

16.5 Applied to navigation

The navigation pipeline can be modeled as:

- IDLE: waiting for a waypoint.
- NAVIGATING: moving toward a target.
- EDIT: applying a waypoint update.
- PAUSED: holding position.
- RETURNING: going back to a visited waypoint.

This structure simplifies reasoning about behavior and failure handling.

CHAPTER

SEVENTEEN

APPENDIX L: PID CONTROL

17.1 Overview

PID control combines proportional, integral, and derivative terms to reduce error between a desired value and the measured value.

17.2 PID equation

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

17.3 Discrete form

$$u_k = K_p e_k + K_i \sum_{i=0}^{k} e_i \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}$$

17.4 Tuning

- K_p sets responsiveness but can overshoot.
- K_i removes steady-state error but can wind up.
- K_d dampens oscillation but amplifies noise.

17.5 Anti-windup

Clamp the integral term when actuators saturate:

$$I_k = \text{clamp}(I_k, I_{min}, I_{max})$$

17.6 Filtering

Derivative terms often use low-pass filtering to reduce noise.

17.7 Applied to robot motion

The controller uses PD components on position and yaw with filtered derivative terms for smoother navigation.

CHAPTER

EIGHTEEN

APPENDIX M: LOCALIZATION

18.1 Overview

Localization estimates a robot's position and orientation in a global frame. It combines motion models with sensor measurements.

18.2 Sources of localization

- Odometry: integrates wheel or motor motion.
- IMU: measures angular velocity and acceleration.
- Visual markers or lidar: provide absolute corrections.

18.3 Probabilistic view

State estimation often uses a probabilistic model:

$$x_k = f(x_{k-1}, u_k) + w_k z_k = h(x_k) + v_k$$

where w_k and v_k represent process and measurement noise.

18.4 Filters

- Extended Kalman Filter (EKF): linearizes nonlinear models.
- Particle Filter: represents state as particles.

18.5 Frames

Common frames in robotics include:

- base_link: robot body.
- odom: local frame for short-term accuracy.
- map: global reference frame.

18.6 Applied to this system

Odometry provides short-term pose estimates. For long-term accuracy, periodic corrections can be applied using external references.

CHAPTER

NINETEEN

APPENDIX N: ODOMETRY

19.1 Overview

Odometry estimates robot motion by integrating velocity over time. It is fast and local but accumulates drift due to wheel slip and noise.

19.2 Kinematic integration

Given linear velocities in the global frame:

$$x_{k+1} = x_k + v_x \Delta t y_{k+1} = y_k + v_y \Delta t$$

Yaw integration uses angular velocity:

$$\theta_{k+1} = \theta_k + \omega_z \Delta t$$

19.3 Error sources

- Wheel slip or uneven terrain.
- Encoder quantization.
- Incorrect wheel radius or geometry parameters.

19.4 Mitigation

- Periodic calibration of wheel parameters.
- Fusing odometry with IMU or external sensors.
- Avoiding long open-loop runs without correction.

19.5 Applied to this project

Odometry is derived from motor velocities and transformed into the global frame for display and control.

CHAPTER

TWENTY

APPENDIX O: WAYPOINT NAVIGATION

20.1 Overview

Waypoint navigation drives a robot through a sequence of target positions. Key components include path planning, control, and arrival detection.

20.2 Path planning

- For sparse waypoints, straight-line segments are common.
- For smoother trajectories, use splines or polynomial paths.

20.3 Arrival detection

A waypoint is considered reached when distance and timing constraints are met:

$$\text{dist} = \sqrt{(x_t - x)^2 + (y_t - y)^2}$$

If dist is below a tolerance for a specified duration, the waypoint is complete.

20.4 Handling edits

When waypoints are edited, smoothing can reduce abrupt changes in command velocity. A simple low-pass update is:

$$x_{new} = (1 - \alpha)x_{old} + \alpha x_{update}$$

20.5 Return behavior

Visited waypoints can be stored in a stack to support a return-to-base function.

20.6 Applied to this system

The app sends waypoint lists and updates to the robot. The controller processes the queue, updates the active target, and transitions between states.

CHAPTER

TWENTYONE

APPENDIX P: COMPLEMENTARY FILTERING

21.1 Overview

A complementary filter combines two signals with different frequency qualities. One signal provides good low-frequency behavior, the other provides high-frequency response. The sum provides a balanced estimate.

21.2 Basic equation

$$y = \alpha y_{fast} + (1 - \alpha) y_{slow}$$

where $0 < alpha < 1$.

21.3 Why it works

- The low-pass component reduces noise and drift.
- The high-pass component preserves responsiveness.

21.4 Discrete implementation

```
filtered = alpha * new_value + (1 - alpha) * previous_filtered
```

21.5 Applied to derivative terms

Derivative signals are noisy. A complementary filter can smooth the derivative term in a PD or PID controller, improving stability without a large delay.

21.6 Applied to waypoint updates

When target updates occur mid-motion, filtering avoids sudden jumps in desired position and keeps the control output smooth.

CHAPTER

TWENTYTWO

APPENDIX Q: CAN BUS

22.1 Overview

Controller Area Network (CAN) is a robust bus protocol used in automotive and robotic systems for reliable device communication.

22.2 Frame structure

- Identifier: arbitration and message priority.
- Data length: number of bytes (0 to 8 for classic CAN).
- Data: payload bytes.
- CRC: error detection.

22.3 Arbitration

CAN uses non-destructive bitwise arbitration. The lowest identifier wins bus access without corrupting messages.

22.4 Bitrate and timing

All nodes must share the same bitrate and timing parameters. Mismatched timing causes errors and bus-off states.

22.5 Error handling

CAN includes error counters and automatic retransmission. Persistent errors can place a node into a bus-off state until recovery.

22.6 Classic CAN vs CAN FD

- Classic CAN: 8-byte payload limit.
- CAN FD: larger payload and higher data phase bitrate.

22.7 Applied to robotics

CAN is often used for motor controllers and sensor modules that require robust communication in noisy electrical environments.

CHAPTER

TWENTYTHREE

INDICES AND TABLES

- genindex
- modindex
- search